

Разделение монолитного backend-приложения на микросервисную архитектуру

1. Назначение документа

Документ дополняет набор OpenAPI-спецификаций и описывает конкретные требования и шаги для перехода от монолитного backend-приложения к микросервисной архитектуре с принципом database-per-service

2. Состав микросервисов и зоны ответственности

Микросервис	Ответственность	БД	Основные таблицы	Важные замечания
Auth/User Service	Пользователи, роли, учетные записи, регистрация, логин, выпуск JWT.	users_db	users	Источник истины для user_id и ролей.
Company Service	Компании и профили работодателей.	companies_db	companies, employer_profiles	employer_profiles.user_id хранится как external ID.
Dictionary Service	Справочники: отрасли и уровни опыта.	dictionaries_db	industries, experience_levels	Используется Vacancy Service для валидации справочников.
Vacancy Service	Вакансии, публикация, владение вакансией работодателем.	vacancies_db	vacancies	company_id, employer_profile_id, industry_id, experience_level_id - external IDs.
Resume Service	Резюме соискателей и опыт работы.	resumes_db	resumes, resume_experiences	resumes.user_id - external ID; resume_experiences.resume_id - внутренний FK.
Application Service	Отклики на вакансии и статусы откликов.	applications_db	applications	vacancy_id, resume_id, user_id - external IDs.
Interaction Service	Избранные вакансии и история просмотров.	interactions_db	favorite_vacancies, vacancy_views	user_id и vacancy_id - external IDs; просмотры хранятся историей.

3. Разделение базы данных по database-per-service

Разделение БД выполняется так, чтобы каждая база имела одного владельца. Другие сервисы не могут выполнять SQL-запросы к чужой БД. Если сервису нужна информация о сущности другого сервиса, он обращается к internal endpoint сервиса-владельца.

БД	Владелец	Таблицы	Правило владения и ссылок
users_db	Auth/User Service	users	Нет внешних зависимостей. Другие сервисы не читают users_db напрямую.
companies_db	Company Service	companies, employer_profiles	employer_profiles.user_id ссылается на пользователя только логически, без FK.
dictionaries_db	Dictionary Service	industries, experience_levels	Справочники доступны через public/internal API, без прямого доступа к БД.
vacancies_db	Vacancy Service	vacancies	company_id, employer_profile_id, industry_id, experience_level_id хранятся без межбазовых FK.
resumes_db	Resume Service	resumes, resume_experiences	resumes.user_id - external ID; допускается внутренний FK resume_experiences.resume_id.
applications_db	Application Service	applications	vacancy_id, resume_id, user_id - external IDs; уникальность отклика проверяется локально.
interactions_db	Interaction Service	favorite_vacancies, vacancy_views	favorite_vacancies может иметь локальный unique(user_id, vacancy_id); vacancy_views хранит историю.

Запрещенные практики после разделения:

- общая база данных для всех сервисов;
- межбазовые foreign keys между таблицами разных сервисов;
- чтение чужих таблиц через SQL JOIN;
- обновление данных другого сервиса напрямую;
- передача бизнес-логики владения сущностями в API Gateway.

4. Матрица межсервисных вызовов

Сервис-инициатор	Сервис-владелец данных	Internal endpoint	Назначение
Vacancy Service	Company Service	GET /internal/v1/companies/{company_id}	Проверка существования компании при создании или обновлении вакансии.
Vacancy Service	Company Service	GET /internal/v1/employer-profiles/{profile_id}	Получение employer profile для проверки связи работодателя с компанией.
Vacancy Service	Company Service	GET /internal/v1/employer-profiles/{profile_id}/ownership	Проверка, что текущий пользователь владеет профилем работодателя.
Vacancy Service	Dictionary Service	GET /internal/v1/industries/{industry_id}	Проверка существования и публикации отрасли.
Vacancy Service	Dictionary Service	GET /internal/v1/experience-levels/{experience_level_id}	Проверка уровня опыта.
Application Service	Vacancy Service	GET /internal/v1/vacancies/{vacancy_id}	Проверка существования вакансии и получение базовых данных.
Application Service	Vacancy Service	GET /internal/v1/vacancies/{vacancy_id}/publication-status	Запрет отклика на неопубликованную вакансию.
Application Service	Resume Service	GET /internal/v1/resumes/{resume_id}	Проверка существования резюме.
Application Service	Resume Service	GET /internal/v1/resumes/{resume_id}/ownership	Проверка, что резюме принадлежит текущему пользователю.
Interaction Service	Vacancy Service	GET /internal/v1/vacancies/{vacancy_id}	Проверка существования вакансии перед добавлением в избранное или фиксацией просмотра.
Frontend/API Gateway	Auth/User Service	GET /users/me	Получение текущего пользователя по JWT.

5. Основные сценарии взаимодействия сервисов

Сценарий	Последовательность действий	Ошибки и бизнес-правила
Создание вакансии	1) Клиент отправляет POST /vacancies. 2) Vacancy Service проверяет JWT и роль employer. 3) Vacancy Service вызывает Company Service и Dictionary Service. 4) После успешных проверок создает запись в vacancies_db.	Если company_id/profile_id/industry_id не найден - 404. Если пользователь не владелец employer profile - 403. Если зависимый сервис недоступен - 503.
Отклик на вакансию	1) Клиент отправляет POST /vacancies/{vacancy_id}/applications. 2) Application Service проверяет JWT. 3) Запрашивает Vacancy Service и Resume Service. 4) Проверяет уникальность отклика. 5) Создает application.	Если вакансия/резюме не найдены - 404. Если резюме чужое - 403. Если отклик уже существует - 409.
Изменение статуса отклика	1) Работодатель вызывает PATCH /applications/{application_id}/status. 2) Application Service проверяет права через vacancy_id и данные владельца вакансии. 3) Статус меняется локально.	Если отклик не найден - 404. Если нет прав - 403. Если статус некорректен - 400.
Добавление в избранное	1) Пользователь вызывает POST /favorites/{vacancy_id}. 2) Interaction Service проверяет вакансию через Vacancy Service. 3) Создает favorite_vacancy.	Если вакансии нет - 404. Если уже в избранном - 409.
Просмотр вакансии	1) Пользователь открывает вакансию. 2) Клиент вызывает POST /vacancies/{vacancy_id}/views. 3) Interaction Service проверяет vacancy_id и сохраняет событие просмотра.	Если вакансии нет - 404. Для просмотра можно не задавать unique constraint, чтобы хранить историю.

6. OpenAPI-спецификации и internal endpoints

Ниже указана сводка по OpenAPI-файлам.

Сервис	Файл	БД	Public endpoints	Internal endpoints
Auth/User Service	auth-user-service.openapi.yaml	users_db	4	1
Company Service	company-service.openapi.yaml	companies_db	9	4
Dictionary Service	dictionary-service.openapi.yaml	dictionaries_db	2	2
Vacancy Service	vacancy-service.openapi.yaml	vacancies_db	6	3
Resume Service	resume-service.openapi.yaml	resumes_db	5	3
Application Service	application-service.openapi.yaml	applications_db	4	3
Interaction Service	interaction-service.openapi.yaml	interactions_db	5	3

6.1. Internal endpoints, необходимые для межсервисного взаимодействия

Сервис	Метод	Endpoint	Назначение	Коды ответов в текущей спецификации
--------	-------	----------	------------	-------------------------------------

Сервис	Метод	Endpoint	Назначение	Коды ответов в текущей спецификации
Auth/User Service	GET	/internal/v1/users/{user_id}	Получить пользователя для межсервисных проверок	200, 404, 503
Company Service	GET	/internal/v1/companies/{company_id}	Internal lookup компании	200, 404
Company Service	GET	/internal/v1/employer-profiles/{profile_id}	Internal lookup employer profile	200, 404
Company Service	GET	/internal/v1/employer-profiles/{profile_id}/ownership	Проверить ownership employer profile	200, 400, 404
Company Service	GET	/internal/v1/employer-profiles/by-user/{user_id}	Найти employer profile пользователя	200, 404
Dictionary Service	GET	/internal/v1/industries/{industry_id}	Получить отрасль для межсервисной проверки	200, 404, 503
Dictionary Service	GET	/internal/v1/experience-levels/{experience_level_id}	Получить уровень опыта для межсервисной проверки	200, 404, 503
Vacancy Service	GET	/internal/v1/vacancies/{vacancy_id}	Internal lookup вакансии	200, 404
Vacancy Service	GET	/internal/v1/vacancies/{vacancy_id}/owner	Получить владельца вакансии	200, 404
Vacancy Service	GET	/internal/v1/vacancies/{vacancy_id}/publication-status	Проверить публикацию вакансии	200, 404
Resume Service	GET	/internal/v1/resumes/{resume_id}	Internal lookup резюме	200, 404
Resume Service	GET	/internal/v1/resumes/{resume_id}/ownership	Проверить владельца резюме	200, 400, 404
Resume Service	GET	/internal/v1/resumes/{resume_id}/publication-status	Проверить публикацию резюме	200, 404
Application Service	GET	/internal/v1/applications/{application_id}	Internal lookup отклика	200, 404
Application Service	GET	/internal/v1/vacancies/{vacancy_id}/applications/count	Получить число откликов на вакансию	200
Application Service	GET	/internal/v1/users/{user_id}/applications	Получить отклики пользователя	200
Interaction Service	GET	/internal/v1/vacancies/{vacancy_id}/favorites/count	Получить число добавлений в избранное	200
Interaction Service	GET	/internal/v1/vacancies/{vacancy_id}/views/count	Получить число просмотров вакансии	200
Interaction Service	GET	/internal/v1/users/{user_id}/favorites	Получить избранное пользователя	200

7. Стандартизация ошибок

HTTP-код	Когда использовать
400 Bad Request	Ошибка валидации входных данных, некорректный UUID, недопустимый статус.
401 Unauthorized	Не передан или некорректен JWT / X-Service-Token.
403 Forbidden	Пользователь или сервис аутентифицирован, но не имеет прав на действие.
404 Not Found	Запрашиваемая сущность не найдена в сервисе-владельце.
409 Conflict	Конфликт бизнес-правила: email уже занят, отклик уже существует, вакансия уже в избранном.
503 Service Unavailable	Зависимый сервис недоступен или истек таймаут межсервисного вызова.

8. План миграции из монолита

Постепенно вытеснить функциональность монолита по pattern Strangler Fig. Сначала выделяются сервисы с минимальным количеством зависимостей, затем сервисы с более сложными бизнес-сценариями.

Шаг	Действие	Описание
1	Инвентаризация монолита	Составить список модулей, таблиц, FK, бизнес-процессов, внешних API и фоновых задач.
2	Выделение Auth/User Service	Перенести users и авторизацию в users_db; все остальные сервисы получают user_id из JWT или через internal API.
3	Выделение Company Service	Перенести companies и employer_profiles в companies_db; заменить FK на users на external ID.
4	Выделение Dictionary Service	Перенести справочники в dictionaries_db; подключить lookup endpoints для Vacancy Service.
5	Выделение Vacancy Service	Перенести vacancies в vacancies_db; убрать межмодульные FK на компании и справочники.
6	Выделение Resume Service	Перенести resumes и resume_experiences в resumes_db; оставить только внутренние FK внутри сервиса.
7	Выделение Application Service	Перенести отклики в applications_db; проверки вакансий и резюме выполнять через internal API.
8	Выделение Interaction Service	Перенести избранное и просмотры в interactions_db; валидировать vacancy_id через Vacancy Service.
9	Введение API Gateway	Сконфигурировать маршрутизацию public endpoints, аутентификацию, rate limit и единый CORS.
10	Настройка межсервисной безопасности	Для /internal/v1/** использовать X-Service-Token, allow-list сервисов, короткие таймауты и аудит.
11	Миграция данных	Перенести данные по сервисам, проверить соответствие external IDs, выполнить сверку количества записей и контрольные запросы.
12	Переключение трафика	Использовать strangler fig pattern: постепенно переводить маршруты с монолита на сервисы, сохраняя rollback-план.
13	Удаление связей с монолитом	После стабилизации убрать старые таблицы/модули, отключить прямой доступ к чужим БД.